Detecting Deepfake Voice Audio Using Adversarial Hidden Markov Models

Jon Montgomery

Abstract

This paper describes a method to detect deepfake audio using adversarial Hidden Markov Models (HMMs) based on the detection model described in the *Advanced Sequential Modeling for Deepfake Audio Identification paper* using Gaussian Mixture Models (GMMs). The model uses two adversarial HMMs trained separately on real and fake audio to achieve effective classification by comparing log-likelihood scores on time-series audio features including Mel-Frequency Cepstral Coefficients (MFCCs), Modified Group Delay Cepstral Coefficients (MGDCCs), spectral contrast, and chroma. The results show a statistically significant ~73% accuracy in classifying real and fake audio files in the test data, suggesting that the model is an effective classifier.

Introduction

The quality of deepfake audio is improving quickly over time. As deepfake audio becomes better and more available, the risk of deepfake scams and attacks increases. An effective classifier could protect people and organizations from spam and attacks involving deepfake audio. GMMs are a popular model for deepfake detection, but they ignore the sequential nature of audio data and could, therefore, ignore abnormalities that become apparent when the features are viewed as a series. Most modern deepfake detection models are based on deep learning, including transformers.

The data used in this experiment are sourced from the Fake-or-Real Dataset on Kaggle. The real data come from the Arctic, LJSpeech, and VoxForge datasets and the fake data come from Google WaveNet TTS and Deep Voice 3 text to speech generation models. The dataset includes 17870 audio files, balanced with equal real and fake files and balanced between genders. All files were clipped at 2 seconds, with normalized sampling rate (16kHz) and normalized amplitude.

R has limited support for audio file formats, so I used Python's Librosa library to extract features from the files. I decided to extract a wide variety of features to maximize my chances of finding dimensions in which the deepfakes deviate from the real audio. All features were computed over 30 ms intervals, resulting in time-series sequences with 67 steps per file (see `extract_features.py`).

Exploratory Data Analysis

The HMM models I built for this classifier estimate the emission probabilities for the features as gaussians, so the features should be well-represented by one or more gaussian distributions.

Figure 1 shows the density plots of the features in the training data. The real data are shown in teal and fake in red.

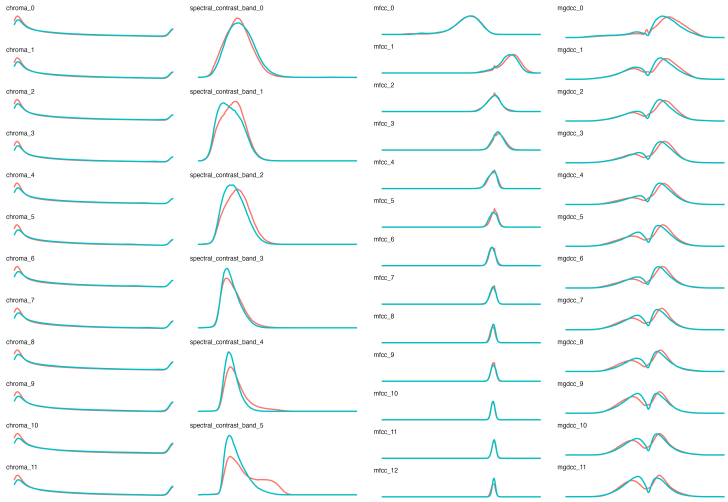

Figure 1: Feature Distributions

Figure 1 shows MGDCC features transformed by taking the quartic root and maintaining the original sign. The two populations differ significantly for certain features.

The chroma features are bimodal distributions with peaks centered near zero and one. This indicates that, within each pitch band, the energy tends to be present or absent. The fake signals appear to have more "zero" values.

The shape of the spectral contrast features differs across the bands, and the fake signals appear to have significant differences from the real features, including different means and different shapes.

The MFCC features are roughly gaussian and the real and fake distributions are closely aligned. There is some separation between the real and fake distributions, presented as abnormal spikes. Where the real signals follow smooth distributions, the fake signals show sharp deviations from the trend in some bands.

The MGDCC features are closely aligned, and present roughly as two overlapping gaussians within each band. The raw MGDCC features are a sharp spike centered around zero, which is poorly modeled by a gaussian distribution. The HMM models fail to converge on emission probabilities for the raw MGDCC features, so I applied the quartic root transformation to cast them onto gaussian shapes.

Methodology

The deepfake prediction model used in this paper is based on adversarial Hidden Markov Models (HMMs). One HMM is trained on real data and the other is trained on fake data. Predictions are made for new sequences by comparing the sequence likelihood for each model.

HMMs model sequences of data with latent states and emissions. Each state emits features drawn from gaussian distributions.The transition matrix models likely state sequences. I used R's `depmixS4` package to create and train the HMMs, and calculate likelihoods.

I chose the number of hidden states empirically by comparing the prediction performance with different numbers of hidden states. The number of hidden states was chosen by optimizing the accuracy of the predictions in the validation set. Once the number of hidden states is chosen, I introduced a balance factor to optimize the F1 score of the model in the validation set with the following formula, where α is the empirically optimized balance factor.

$$\text{prediction}(x) = \begin{cases} \text{fake} & \text{if } \alpha \times \text{likelihood}_{fake}(x) > \text{likelihood}_{real}(x) \\ \text{real} & \text{else} \end{cases}$$

HMMs are a good choice for modeling audio data because they are sequential, and the gaussian emissions match the empirical distributions of the extracted features fairly well. HMMs assume that the observed features are independent from each other, and that the hidden states follow the markov assumption.

The independence of the observed features is questionable and the markov assumption is hard to verify. We will take these assumptions as true for the purpose of training the models and making predictions, but they may not be strictly true.

Results

I found that the optimal number of hidden states was four.

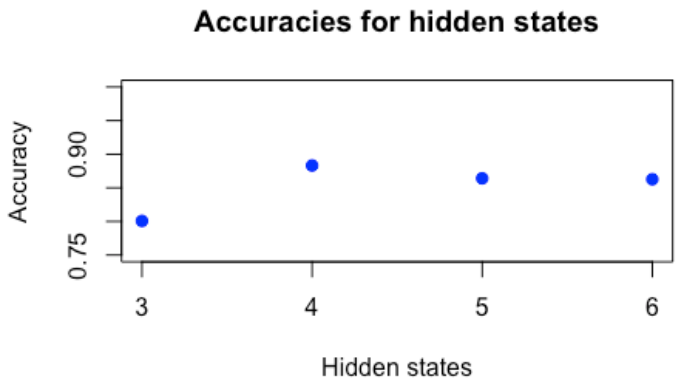

Figure 2: Hidden States

Figure 2 shows the accuracies of models on the validation data with different numbers of hidden states.

Choosing HMMs with four hidden states, I noticied trained the balance factor to optimize the F1 score of the model on the validation data. As α increases, the accuracy of the model to predict fake files monotonically increases and vice versa. This is a useful property because it allows us to tune the predictions to our desired outcome. If we prefer to generate false positives or false negatives, we can modify the balance factor to achieve the optimal balance. In this experiment, I chose to optimize the F1 score for performance that balances precision and recall.

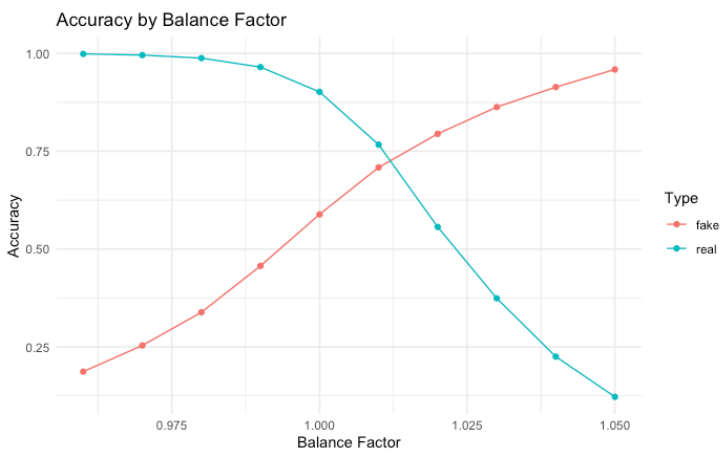

Figure 3: Accuracies

Figure 3 shows the accuracies of the model on the real and fake validation data for different balance factors. The F1 score is optimized where the lines intersect, as shown in Figure 4.

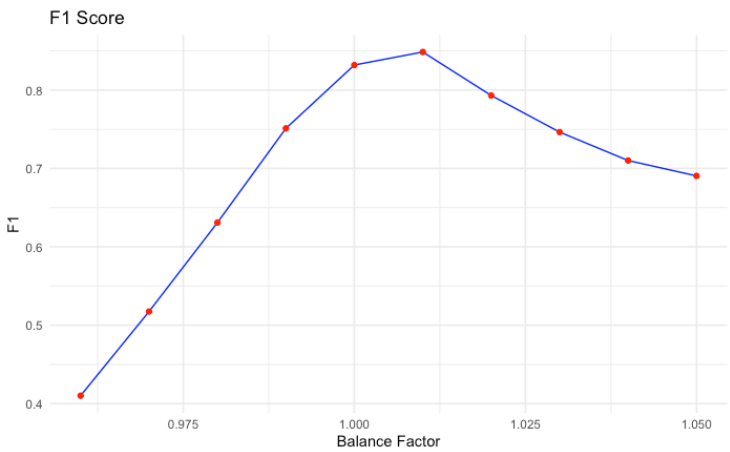

Figure 4: F1 Score Optimization

The F1 score is optimized for $\alpha = 1.01$. The optimal F1 score achieved on the validation data was 0.85. When the model was applied to the testing data, it achieved an F1 score of 0.73.

The significance of the predictions were compared to the no-information-rate with a binomial test with $p = 0.5$. The p-value of the binomial test was $< 2.2 \times 10^{-16}$, indicating that the predictions are statistically significant.